

Economic Denial of Sustainability (EDoS) Detection using GANs in SDN-based Cloud

Phuc Trinh Dinh (Member, IEEE) ¹, and Minh Park (Member, IEEE) ^{*1,2}

¹Department of Information Communication, Materials, and Chemistry Convergence Technology, Soongsil University, Seoul 156-743, South Korea

²School of Electronic Engineering, Soongsil University, Seoul 156-743, South Korea
phuctrinh@soongsil.ac.kr, and mhp@ssu.ac.kr

Abstract—Cloud computing is now considered to be the most cost-effective platform for offering business and consumer IT services over the Internet. However, it is prone to new vulnerabilities. Specifically, a newly discovered type of attack, called an economic-denial-of-sustainability attack known as EDoS, exploits the pay-per-use model to scale up the resource usage over time to the degree that the cloud user has to pay for the unexpected usage charge. To prevent EDoS attacks, we propose an effective solution in the SDN-based cloud computing environment. We first introduce a machine-learning-based approach adopting a framework called MAD-GAN which applies an unsupervised multivariate anomaly detection technique based on Generative Adversarial Networks (GANs), using the Long-Short-Term-Memory Recurrent Neural Networks (LSTM-RNN) to detect EDoS attacks. Its main idea is to produce an anomaly score at each time step by learning a multivariate attribute. We then generate a dynamic threshold score to compare with the anomaly score produced to classify the network traffic as EDoS traffic or normal traffic. By realistic tests, our proposed scheme is demonstrated to outperform existing methods for EDoS attack detection. The detailed experiments conducted with different levels of EDoS attacks show that the proposed scheme is an efficient, innovative approach to defend EDoS attacks in the SDN-based cloud.

Index Terms—Cloud computing security, economic denial of sustainability (EDoS), software-defined networking, machine learning, network function virtualization.

I. INTRODUCTION

Cloud Computing provides some of the unique features such as usage-based billing, elasticity, auto scalability, which are considered as a significant advantage over conventional computing services. However, Cloud Computing is rather vulnerable to some particular attacks [1]. Due to a pay-per-use pricing system, EDoS [2], a novel variant of Distributed Denial of Service (DDoS) attacks [3], is emerging as one of the biggest challenges.

A cloud service provider typically automatically scales up a customer's resources to meet the Service Level Agreements (SLA) for the customer's service quality and charges the customer for the use. EDoS attacks exploit this feature to render the cloud unsustainable by disrupting the cloud billing engine to bill the cloud user for the attack's activities. Since EDoS is relatively new and very stealthy, it is tougher to discover EDoS behaviors compared to DDoS. Therefore, only

a few existing proposals have a satisfactory solution to deal with the problem [2].

Some of the existing mitigation approaches [4], [5], [7] focused exclusively on either crypto-puzzle-based verifier or graphical Turing test to distinguish human from machine. However, such techniques often trigger time complexity which is very difficult to deploy in real life. Despite these techniques, many researchers employed a predefined threshold value setting which may lead to a higher false-negative rate [2]. Recently, a few new approaches such as the queuing model, statistical have been applied to diminish EDoS attacks [8], [9]. However, according to the authors, their approaches primarily still have their own drawbacks such as lacking evaluation metrics and generating high rate errors. Understanding these difficulties, in our paper, we propose a scheme to detect EDoS attacks effectively, which applies the combination of the two well-known methods which are GANs and LSTM-RNN. This combination leverages the multivariate time-series anomaly detection ability of such a stealthy attack as EDoS [2]. In addition, unlike other current mitigation techniques [4]–[7], [21] focused solely on setting a predefined statics threshold value which rises false-error rates [8], in this paper, we introduce a dynamic error threshold technique to dynamically set the appropriate threshold. Our major contributions are as follows:

- We propose an effective and innovative hybrid scheme to detect effectively such a stealthy attack as EDoS by leveraging the combination of GANs and LSTM-RNN, and then utilizing the Dynamic Error Threshold to dynamically set the optimal threshold to differentiate between normal and abnormal traffic.
- We study common types of EDoS attacks in traditional clouds and introduce a new viewpoint of EDoS attacks in the SDN environment.
- Our experiments are conducted in a real cloud computing environment using OpenStack [20] integrated with the SDN environment using ONOS controller [24], but it can be easily applied to other controllers.

The rest of the paper is structured as follows: In section II, related work is presented. we quickly introduce the background knowledge in section III. We describe in detail our proposed scheme in section IV and experimental setup in

* Corresponding author

section V. Next, in section VI, we evaluate our mechanism with detection rate, False Alarm Rate, accuracy, and performance. Lastly, in Section VII, conclusion and future researches are presented.

II. RELATED WORK

A variety of studies on EDoS mitigation have been performed since the security of EDoS cloud clients has become crucial. EDoS-Shield [4] is a mitigation method first proposed by Squalli et al.. Two main components, virtual firewall (VFs) and authentication nodes (V-nodes) play a crucial role to detect EDoS attacks. Even though EDoS-Shield shows its practical but, nonetheless, there are major drawbacks of the method proposed. Particularly, it is easily susceptible to spoofing attacks and possesses a higher false-positive rate by blocking many legitimate user IP addresses since the two lists are not updated in an effective manner. sPoW [5] is another EDoS mitigation technique which its core function is to filter illegitimate traffic. There are a few defects in the scheme. For example, an attacker can effortlessly start an attack without aiming to solve crypto-puzzle based request and the scheme can be easily prone to false-positive rates as some of the normal users cannot send requests to the service owing to rising puzzle complexity. EDoS Armor [6] consists of two systems (congestion and admission control). The goal of two systems is to limit the number of end-users and to set priorities to the client based on their browsing history. Overall, EDoS Armor is quite effectual; however, the adaptability of the model needs to take into account because potential new users may lose interest in the site if it is too complex. Baig *et al.* proposed a scheme that be comprised of two phases to mitigate EDoS which are blacklist and Turing Test to detect suspected users [21]. Although the scheme may work for some particular attacks, it becomes useless once it faces a botnet attack which makes the blacklist overload.

Recently, a few new approaches such as the queuing model, statistical analysis have been applied to tackle EDoS attacks. A new solution is known as EDoS Eye employing game theory to mitigate the invalid traffic [8]. The authors attained optimal threshold value using the Nash equilibrium and incorporated honeypot to minimize false rates. Another EDoS detection scheme proposed by Karami & Chen [9], in which an anomaly-based detection technique called Markov models were obtained. However, according to the authors, the technique only aimed at identifying EDoS traffic instead of mitigation.

III. BACKGROUND KNOWLEDGE

A. EDOS ATTACKS IN SDN-BASED CLOUD

To prevent EDoS attacks effectively, we understand that it is important to know the characteristics of EDoS. For this reason, we briefly compare EDoS characteristics in the cloud with DDoS. In the DDoS attack, the attacker's purpose is to entirely shut down the services provided by a cloud provider by irrationally launching an attack with their utmost resources. Conversely, attackers are usually cleverer and highly logical in

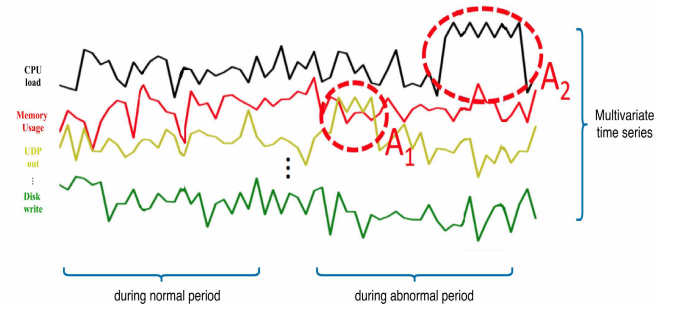


Figure 1: Multivariate time series snippet visualization, with two anomalies A_1 and A_2 marked by red dash circle. The root causes are yellow and black time series, respectively

attacks with EDoS. They specifically either target a specific group or an individual of cloud users. They gradually send unauthorized traffic over a longer period of time and their ultimate goal is to make the targeted cloud user paying more. EDoS is incredibly stealthy in nature and always tries to stay below a traditional intrusion detection system's threshold value so that it can pass the system as legitimate traffic.

For the viewpoint of SDN, which is a flow-based rule network system, types of EDoS attack are listed in Table I. These volumetric-based attacks not only make cloud user but also SDN controller or Virtual Network Function consuming a lot of resources (e.g. CPU, RAM and so on).

B. BASICS OF GANs & LONG SHORT-TERM MEMORY RECURRENT NEURAL NETWORKS

GANs [11] is a machine learning algorithm, in which two neural networks compete with each other to become more accurate in their predictions. GANs can solve the issue of dataset lack and can provide a quick classification of network anomalies.

Recurrent neural network (RNN) [12] where the output from the previous step is fed as input to the current step and LSTM [13] are often present in multivariate time-series data streams. Both RNN and LSTM obviate the need to specify a time-window; therefore, their combination is capable of learning complex time series. Moreover, GAN-based anomaly detection can leverage the ability to capture the temporal correlation of time series distributions.

IV. A PROPOSED EDOS ATTACK DEFENSE

In this section, we first introduce our objective. Next, the workflow system is presented. Finally, the internal modules is explained in detail.

TABLE I: EDoS attacks in SDN-Based Cloud

Attack	EDoS Simulator
TCP SYN slow-flooding attack	Bonesi
TCP (HTTP) slow-flooding attack	Bonesi
UDP slow-flooding attack	Bonesi
Flow table slow-flooding attack	slow_ftf.py
Slowloris	slowloris.py

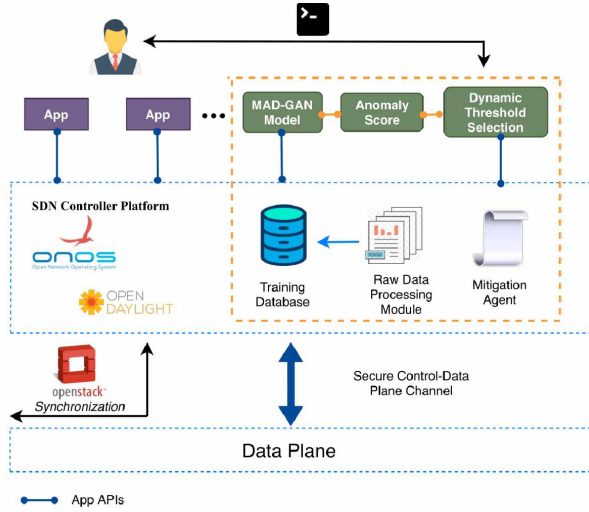


Figure 2: Conceptual Architecture

A. AN OBJECTIVE AND SYSTEM DESIGN ANALYSIS

In the SDN-based cloud, cloud consumers are normally tracked with multivariate time series, whose anomaly detection is crucial for service quality management. For example, as shown in Figure 1, networking metrics such as Memory usage, CPU load, TCP connections and etc are supervised by a network administrator, and when an EDoS attack is launched, one of the selected features will suddenly change their values. Thus, we need an intelligent model to discover any changes in values of multiple time series simultaneously to detect EDoS. Therefore, in this paper, a framework called MAD-GAN [17] is adopted for multivariate time series anomaly detection. By learning from historical data and then using live data to predict future values, we compare the values with a dynamic error threshold to determine anomalies. Therefore, we claim that our proposed scheme is very a practical and effective method to detect EDoS attacks.

Figure 2. depicts an overview of our proposed framework aimed to detect EDoS attack in Software-Defined Networking architecture synchronized with OpenStack [20] controller and it is an extended module in the Control Plane. This framework consists of additional modules that can be deployed and distributed both in the cloud controller platform (OpenStack) and SDN application plane. We recommend that these modules should be placed at a dedicated security server in real deployment in order to reduce the SDN controller processing load. For convenience, however, in this work we design modules including MAD-GAN Model, Dynamic Error Threshold and Anomaly Score Classifier and site these modules in the SDN application layer, only a few components are located in the OpenStack controller for synchronization. Also, bear in mind that, our proposed scheme can be deployed in a distributed manner of nowadays cloud computing, in which databases can be shared among distributed agents to enhance the efficiency of anomaly detection.

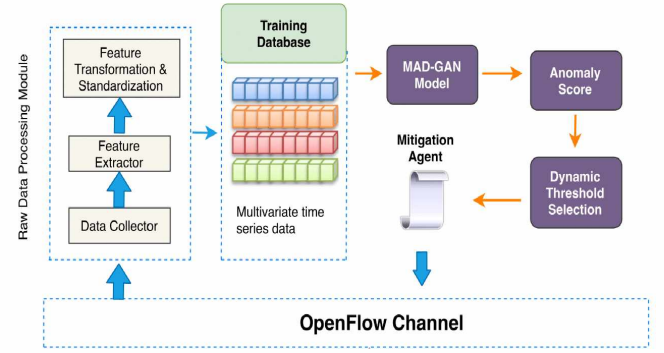


Figure 3: Detailed architecture modules synchronized OpenStack controller located in the SDN application layer

B. A WORKFLOW SYSTEM

In our work, the mechanism is situated in the SDN controller and comprises of six separate modules: Raw Data Processing (Flow Collector, Feature Extractor and Feature Transformation and Normalization), MAD-GAN Model, Anomaly Result, Dynamic Threshold Selection, and Mitigation Agent as shown in Figure 3. In the data preprocessing stage, the Data Collector reliably collects data on virtual resources comprising deployed clouds and the utilization of the physical and accumulates flow information from OpenFlow switches in a predetermined interval time. Afterward, this data is sent to the Feature Extractor to extract data's attributes. Next, the data is transformed by data standardization, and then it is saved in the Training Database module. After the preprocessing, the stored data which represents a multivariate time-series representation in the Training Database is sent to the MAD-GAN Model module to get anomaly scores. Also, the Dynamic Threshold Selection module sets an anomaly threshold dynamically. By comparing the anomaly score with the threshold, the network traffic will be classified as anomalous or normal. If the traffic is declared as normal traffic, then it will be normally forwarded to the destination; if not, the traffic will be transferred to the Mitigation Agent which performs a dropping or deleting action if it detects abnormal traffic.

Remember that, in the proposed framework, the Training Database is always updated by data's attributes collected from the above loop. In a preset time which is defined by a network administrator, the MAD-GAN Model will be trained using the updated database. By doing so, our proposed framework can well adapt to the various network systems.

C. INTERNAL MODULES

Let us have a closer look into the internal modules of our proposed mechanism.

- **DATA COLLECTOR:** this module simply runs in the SDN controller. It collects data from both OpenFlow statistics and the utilization rate of the physical and virtual resources comprising deployed clouds.
- **FEATURE EXTRACTOR:** it extracts data information from Data Collector to take out attributes, as shown

in Table II. These features are key features which are selected by selecting k highest scores based on chi-squared statistic [14], in which the Key features are measured in the SDN-based cloud.

- **FEATURE TRANSFORMATION & STANDARDIZATION:** at this stage, all the categorical features, which are extracted in the previous stage, are transformed from categorical data into numeric using One-Hot encoding technique. After the transformation, the data needs to be normalized, and its formula can be expressed as:

$$z = \frac{x - \mu}{\sigma} \quad (1)$$

where x is the value that is standardized, μ is the mean of the distribution and σ is the standard deviation of the distribution. This module not only significantly improves the MAD-GAN Model's predictive power, but also offers them for faster to run and more easily understood.

- **MAD-GAN MODEL:** Given a training dataset $\chi \subseteq \mathcal{R}^{M \times T}$ and a testing dataset $\chi^{test} \subseteq \mathcal{R}^{N \times T}$ with T streams and M, N measurements for each stream, the task is to attempt to distinguish the normal traffic from anomalous traffic in testing dataset. Note that we assume all the traffic in the training dataset are legitimate. To learn from X effectively, we obtain a sliding window with window size s_w and step size s_s to divide the multivariate time series into a set of multivariate sub-sequences $X = x_i, i = 1, 2, \dots, m \subseteq \mathcal{R}^{s_w \times T}$ [17], where $m = \frac{(M-s_w)}{s_s}$ is the number of sub-sequences. Similarly, $Z = z_i, i = 1, 2, \dots, m$ is a set of multivariate sub-sequences taken from a random space. By feeding X and Z to the GAN model, we train the generator and discriminator with the following two-player minimax game:

$$\begin{aligned} \min_G \max_D V(D, G) = & \varepsilon_x p_{data}(X) [\log D(x)] \\ & + \varepsilon_z p_z(S) [\log(1 - D(G(z)))] \end{aligned} \quad (2)$$

In [17], both the generator (G) and discriminator (D) of GAN are Long Short Term-Recurrent Neural Networks. After sufficient rounds of training iterations, the trained discriminator D_{rnn} and the generator G_{rnn} can then be employed to detect anomalies in χ^{test} using a combined Discrimination and Reconstruction Anomaly

TABLE II: Key Features

Feature	Description
protocol_type	network protocol
no. of bytes	number of bytes
no. of packets	number of packets
flow_duration	flow duration
network_bandwidth	network bandwidth
duration	length (number of seconds) of the connection
cpu_load	CPU usage
memory_usage	memory usage
src_ip	source IP address
dst_ip	destination IP address

Score (DR-Score). For detection, the testing dataset $\chi^{test} \subseteq \mathcal{R}^{\mathcal{R} \times \mathcal{T}}$ is similarly divided into multivariate sub-sequences $X^{tes} = x_j^{tes}, j = 1, 2, \dots, n$ with a sliding window, where $n = \frac{(N-s_w)}{s_s}$. Using the computed DR-Score (DRS) of the testing dataset, we label each of the sub-sequences in the testing dataset as follows:

$$A_i^{tes} = \begin{cases} 1, & \text{if } H(DRS_t, 1) > \tau \\ 0, & \text{else} \end{cases} \quad (3)$$

where $A_i^{tes} \subseteq \mathcal{R}^{N \times 1}$ is the outcome vector for the testing dataset, where non-zero values indicate an anomaly is detected (if the cross-entropy error $H(DRS_t, 1)$ for the anomaly score is higher than a predefined value τ). Also, the residual at time t is calculated as:

$$e^t = Res(X_t^{tes}) = \sum_{i=1}^n |x_t^{tes,i} - G_{rnn}(Z_t^{k,i})| \quad (4)$$

where $X_t^{tes} \subseteq \mathcal{R}^n$ is the measurements at time step t for n variables. Finally, the prediction error is expressed as:

$$\mathbf{e} = [e^{(t-h)}, \dots, e^{(t-l_s)}, \dots, e^{(t-1)}, e^{(t)}]$$

where h is the number of historical error values used to evaluate current errors.

- **DYNAMIC THRESHOLD SELECTION:** Unlike other existing works that heavily relied on a hard threshold, in our work, we adopt a technique that can dynamically set the threshold effectively [15]. With a threshold ϵ selected from the set:

$$\epsilon = \mu(\mathbf{e}_s) + \mathbf{z}\sigma(\mathbf{e}_s)$$

where $\mathbf{e}_s$ is smoothed by using an exponentially-weighted average (EWMA) [16], and ϵ is determined by:

$$\epsilon = \operatorname{argmax}(\epsilon) = \frac{\Delta\mu(\mathbf{e}_s)/\mu(\mathbf{e}_s) + (\Delta\sigma(\mathbf{e}_s)/\sigma(\mathbf{e}_s))}{|\mathbf{e}_a| + |\mathbf{E}_{seq}|^2}$$

such that:

$$\Delta\mu(\mathbf{e}_s) = \mu(\mathbf{e}_s) - \mu(\{e_s \in \mathbf{e}_s | e_s < \epsilon\})$$

$$\Delta\sigma(\mathbf{e}_s) = \sigma(\mathbf{e}_s) - \sigma(\{e_s \in \mathbf{e}_s | e_s < \epsilon\})$$

$$\mathbf{e}_a = \{e_s \in \mathbf{e}_s | e_s > \epsilon\}$$

$$\mathbf{E}_{seq} = \text{continuous sequences of } e_a \in \mathbf{e}_a$$

- **ANOMALY SCORE:** to get the anomaly score, the DR-Score [17] is calculated as:

$$DRS_t = \frac{\sum_{j,s \in \{j+s=t\}} L_{j,s}}{lc_t}$$

$$lc_t = \text{count}(j, s \in \{j + s = t\})$$

where $t \in \{1, 2, \dots, N\}, j \in \{1, 2, \dots, n\}, s \in \{1, 2, \dots, s_w\}$, and the anomaly detection loss is $L_t^{tes} = \lambda Res(X_t^{tes}) + (1 - \lambda) D_{rnn}(X_t^{tes})$ [17].

- **MITIGATION AGENT:** if the traffic is considered abnormal, then, in order to mitigate the EDoS attack,

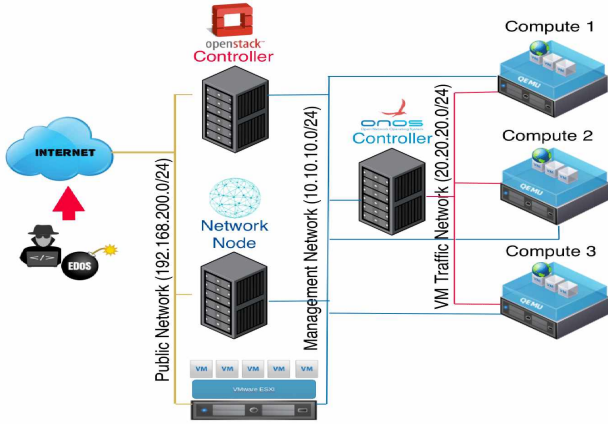


Figure 4: Experimental topology

the mitigation Agent sends a *flow_mod* message with a *delete* action to the edge OpenFlow switch and requests the forwarding engine of the SDN controller to drop *packet_in* messages of attacking sources (as illustrated in Figure 3).

V. EXPERIMENTAL SETUP

Server Machine Dataset (SMD) is obtained to train the MAD-GAN model. This dataset was collected from a large-sized Internet company, and it was publicly published in [22]. In the SMD, the training & testing set size is 58317 and 73729, respectively; in which the anomaly ratio occupies 46.11% and 39.42% for the training and testing set, respectively. To train the MAD-GAN, we strictly follow the instructions in [17].

In this paper, we followed and implemented OpenStack-SDN based test-bed for EDoS as proposed in [19]. Figure 4 shows the testbed for virtual networks to simulate EDoS attack in production environment which includes an OpenStack platform (one Controller node, one Network node, one VMware ESXI server to create multiple VMs, and three Compute nodes running OvS drivers for cloud networking), an SDN controller (ONOS), the Internet connection and botnets. The victim machine was injected with different kinds of EDoS attacks using different simulators (as shown in Table I, in which *slow_flf.py* and *slowloris.py* are coded by us), besides sFlow-RT was obtained to gather data. As we stated in section IV.a, our modules locate in the SDN application layer of the ONOS server (Figure 2). The ONOS server system configurations are dodeca-core CPU (Intel(R) Core(TM) i7-8700 CPU @ 3.20GHz) and a total 48GB memory, running 64 bit Ubuntu Linux v18.04. Note that, we will evaluate our scheme on this server. In the proposed algorithm, attack times are 8 minutes long, in which slow-flooding attacks were conducted in 6 minutes long.

VI. PERFORMANCE EVALUATION

In this section, we will compare our EDoS attack defender with another anomalous machine-learning-based detection algorithm, which is One-Class SVM [23]. This algorithm is unsupervised learning, which is similar to our algorithm;

however, it is a simpler technique and is easy to deploy; therefore, One-Class SVM is chosen. Also, to the best of our knowledge, the work proposed in [21] is the only existing work that evaluates their mechanism using two evaluation metrics: Response Time and CPU Usage which are essential criteria to evaluate the EDoS defense mechanism [2]. Hence in this paper, our proposed scheme will be compared with Zubair A.Baig et al. 's proposal.

In order to properly evaluate the performance of our machine-learning-based scheme, proper criteria for assessing such unsupervised-learning methods including Detection Rate, Accuracy, and False Alarm Rate [18] are necessary. The criteria are calculated by using 4 distinct measurements, True Positive (TP), True Negative (TN), False Positive (FP) and False Negative (FN). Specifically, True Positive is the probability of abnormal traffic which is classified as illegitimate traffic, True Negative shows the probability of legit traffic that is trusted as legitimate traffic, False Positive reflects the probability of normal traffic that is accepted as abnormal traffic and False Negative reveals the probability of illegal traffic that are recognized as legit traffic. Besides, Response Time is also an important aspect to consider a good EDoS attack detection mechanism [2].

- Detection Rate (Sensitivity/TPR): is the ratio of the number of detected malicious flows to the number of all malicious flows: $TPR = \frac{TP}{TP+FN}$
- Accuracy (AC): shows the percentage of true detection over total the number of flows: $AC = \frac{TP+TN}{TP+TN+FP+FN}$
- False Positive Rate (FPR): is the proportion of abnormal flows incorrectly identified as normal flows: $FPR = \frac{FP}{FP+TN}$

Based on the above formulas and Table III, the Detection, Accuracy, and False Alarm Rate are calculated respectively as illustrated in Figure 5.a. Here, regarding the Detection Rate and Accuracy, our novel approach outperforms the One-Class SVM mechanism when they always account for the highest rate, 95.11%, and 94.84%, respectively. Whereas, the One-Class SVM respectively occupies 84.88% and 86.22% in term of both the ratio of detection and the accuracy. In respect of the False Alarm Rate, our scheme also presents domination in producing the wrong warning when only occupies 5.44%, while the rate of incorrect alarm generated by the One-Class SVM is around 7.36%. Overall, the One-Class SVM algorithm not only produces a lower detection rate and accuracy but also causes a higher false alarm rate.

Next, in Figure 5.b, owing to the lightweight of the Dynamic Error Threshold module of our mechanism, the end to end response time of our proposed scheme is less, which only slightly fluctuates between 22ms and 28ms. However, due to a simpler design, One-Class SVM performs better than our

TABLE III: Confusion Matrix

TP (%)	TN (%)	FP (%)	FN (%)
94.53	95.14	5.47	4.86

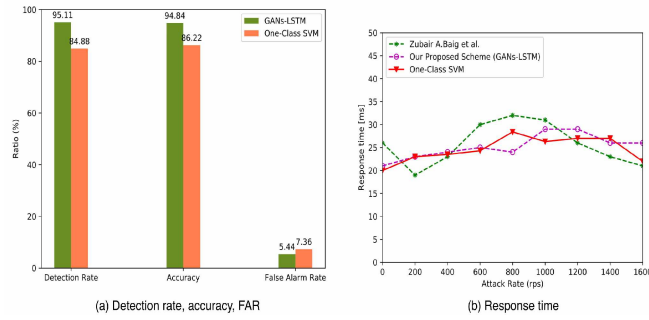


Figure 5: Performance Evaluation

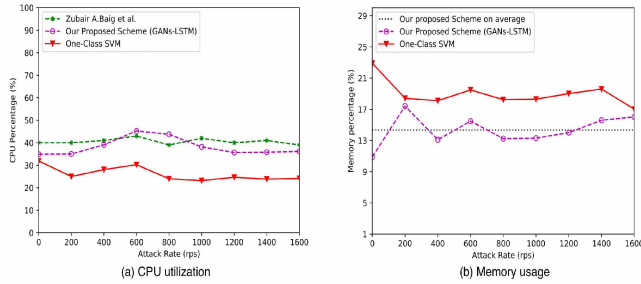


Figure 6: CPU and memory usage

proposed scheme a bit. Whereas, the Zubair A.Baig et al' work creates much higher response time. Figure 6.a and 6.b reveal the CPU usage and Memory usage comparison, respectively. The percentage of CPU usage of our proposed scheme and Zubair A.Baig et al' work are around 37.88%, 41.33%, respectively. With respect to memory utilization, our proposed scheme only consumes around 14.11% of the memory on average, meanwhile, the One-Class method occupies much higher memory on average compared to our mechanism.

Overall, our proposed scheme outperforms both the Zubair A.Baig et al' proposal and the One-Class SVM.

VII. CONCLUSION AND FUTURE WORK

In this paper, we propose a state-of-the-art approach to detect and mitigate EDoS attacks in the SDN-Based cloud environment. The evaluation in section VI shows that our proposed scheme is very efficient in both terms of accuracy and resource consumption. As our future work, we expect to improve the mechanism and compare our proposal with other existing works and with different GAN-based methods using more evaluation criteria.

ACKNOWLEDGMENT

This work was partly supported by the National Research Foundation of Korea(NRF) grant funded by the Korea government(MSIT) (No. 2020R1F1A1076795), and by Institute for Information & communications Technology Promotion(IITP) grant funded by the Korea government(MSIT) (No.2018-0-00254, SDN security technology development).

REFERENCES

- [1] Q. Yan, F. R. Yu, Q. Gong and J. Li, "Software-Defined Networking (SDN) and Distributed Denial of Service (DDoS) Attacks in Cloud Computing Environments: A Survey, Some Research Issues, and Challenges," vol. 18, no. 1, pp. 602-622, Firstquarter 2016.
- [2] F. Z. Chowdhury, L. B. M. Kiah and M. A. M. Ahsan, "Economic denial of sustainability (EDoS) mitigation approaches in cloud: Analysis and open challenges," 2017 ICECOS, Palembang, 2017, pp. 206-211.
- [3] T. V. Phan, N. K. Bao and M. Park, "A Novel Hybrid Flow-Based Handler with DDoS Attacks in Software-Defined Networking," 2016 Intl IEEE, Toulouse, 2016, pp. 350-357.
- [4] M. H. Sqalli, F. Al-Haidari and K. Salah, "EDoS-Shield - A Two-Steps Mitigation Technique against EDoS Attacks in Cloud Computing," 2011 UCC, Victoria, NSW, 2011, pp. 49-56.
- [5] Khor, S.H. and A. Nakao, "spow: On-demand cloud-based eddos mitigation mechanism," in HotDep (Fifth Workshop on Hot Topics in System Dependability). 2009.
- [6] M. Masood, Z. Anwar, S. A. Raza and M. A. Hur, "EDoS Armor: A cost effective economic denial of sustainability attack mitigation framework for e-commerce applications in cloud environments," INMIC, Lahore, 2013, pp. 37-42.
- [7] M. N. Kumar, P. Sujatha, V. Kalva, R. Nagori, A. K. Katukojwala and M. Kumar, "Mitigating Economic Denial of Sustainability (EDoS) in Cloud Computing Using In-cloud Scrubber Service," 2012 Fourth CICN, Mathura, 2012, pp. 535-539.
- [8] F. Z. Chowdhury, M. Y. I. Idris, M. L. M. Kiah and M. A. M. Ahsan, "EDoS eye: A game theoretic approach to mitigate economic denial of sustainability attack in cloud computing," 2017 ICSGRC, Shah Alam, 2017, pp. 164-169.
- [9] M. Karami, S. Chen, "Attribution of economic denial of sustainability attacks in public clouds," In EAI SecureComm, 2016.
- [10] Junyoung Chung, C. Gulcehre, K. Cho, Y. Bengio, "Empirical Evaluation of Gated Recurrent Neural Networks on Sequence Modeling," ArXiv Preprint <https://arxiv.org/abs/1412.3555>, 2014.
- [11] I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, et al. Generative adversarial nets. In Advances in Neural Information Processing Systems, pages 2672-2680, 2014.
- [12] Pankaj Malhotra, L. Vig, G. Shroff, P. Agarwal, "Long short term memory networks for anomaly detection in time series", ESANN 2015 proceedings, Computational Intelligence and Machine Learning.
- [13] O. P. Ogunmolu, X. Gu, S. B. Jiang, and N. R. Gans, "Nonlinear systems identification using deep dynamic neural networks," arXiv preprint arXiv:1610.01439, 2016.
- [14] A. Tamilarasan, S. Mukkamala, A. H. Sung, K. Yendrapalli, "Feature Ranking and Selection for Intrusion Detection Using Artificial Neural Networks and Statistical Methods," The 2006 IEEE IJCNN, Vancouver.
- [15] K. Hundman, V. Constantinou, C. Laporte, I. Colwell, Tom Soderstrom, "Detecting Spacecraft Anomalies Using LSTMs and Nonparametric Dynamic Thresholding," In Proceedings of ACM Knowledge Discovery and Data Mining (KDD'18). ACM, New York, NY, USA.
- [16] J. Stuart Hunter, "The exponentially weighted moving average," Journal of quality technology, 1986 - Taylor & Francis.
- [17] D. Li, D. Chen, B. Jin, L. Shi, J. Goh, and S.-K. Ng, "MAD-GAN: Multivariate anomaly detection for time series data with generative adversarial networks," in Proc. Int. Conf. Artif. Neural Netw. Cham, Switzerland Springer, 2019, pp. 703-716.
- [18] N. Goix, "How to evaluate the quality of unsupervised anomaly detection algorithms?" arXiv preprint arXiv:1607.01152, 2016.
- [19] P. Singh and S. Manickam, "Design and deployment of OpenStack-SDN based test-bed for EDoS," 2015 4th ICRITO, Noida, 2015, pp. 1-5.
- [20] OpenStack: Open Source Cloud Computing Software, Openstack.org, 2016. [Online]. Available: <https://www.openstack.org/>.
- [21] Baig, Z.A., S.M. Sait, F. Binbeshir, "Controlled access to cloud resources for mitigating Economic Denial of Sustainability (EDoS) attacks," Computer Networks, 2016, 97: p. 31-47.
- [22] Y. Su, Y. Zhao, C. Niu, R. Liu, W. Sun, D. Pei, "Robust anomaly detection for multivariate time series through stochastic recurrent neural network," in Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, ser. KDD '19.
- [23] Kun-Lun Li, H. Huang, S. Tian and Wei Xu, "Improving one-class SVM for anomaly detection," Proceedings of the 2003 ICMLC.
- [24] P. Berde et al., "ONOS: Towards an open, distributed SDN OS," in Proc. 3rd Workshop Hot Topics Softw. Defined Netw., 2014, pp. 1-6.